Information Security

Assignment 01 – Fall 2026

Breaking PsycoSupport

Al-Kindi Frequency Analysis of an Affine-Cipher Client–Server Channel

Syed Muhammad Faateh Haneef

23i-0688

22 October 2026

Contents

Summary **2**

1 Ciphertext Collection & Evidence **2**

1.1 Offline lab environment . . . 2
1.2 Capture method . . . 3
1.3 Test cases . . . 4
1.4 Message format on the wire . . . 4
1.5 Ciphertext inventory . . . 4
1.6 Capture evidence . . . 5

2 Frequency Analysis & Key Recovery **6**

2.1 Method . . . 6
2.2 Server → Client (psychologist replies) . . . 7
2.3 Client → Server (patient requests) . . . 10
2.4 Key schedule . . . 11

3 Decryption Results **11**

3.1 Worked example: Server → Client (S1, mood 5) . . . 11
3.2 Worked example: Client → Server (C2, mood 2) . . . 11
3.3 All ciphertexts . . . 12
3.4 Validation: an Ollama-backed `server.py` . . . 12

4 Security Analysis & Reflection **13**

4.1 Why LLM output makes a classical cipher easy to break . . . 13
4.2 Design flaws beyond the cipher . . . 13
4.3 Recommended alternatives (implemented and tested) . . . 14
4.4 Reflection . . . 15

5 Submitted Files & Environment **16**

References **17**

A Full decrypted replies **18**

B Frequency tables and charts **20**

Summary

The PsycoSupport client and server were deployed fully offline on Linux, five patient sessions were sent, and all traffic on the loopback interface was captured. This produced **10 ciphertexts** (5 in each direction). Using Al-Kindi frequency analysis, every affine key was recovered from the ciphertext alone and **all 10 ciphertexts were decrypted**.

Table 1: Recovered keys. The same key is used in both directions for a given mood.

Mood	(a, b)	$a^{-1} \bmod 26$	Sessions	Decryption rule
2	(5, 4)	21	2	$D(y) = 21(y - 4) \bmod 26$
5	(11, 10)	19	1, 3	$D(y) = 19(y - 10) \bmod 26$
8	(17, 16)	23	4	$D(y) = 23(y - 16) \bmod 26$
10	(21, 20)	5	5	$D(y) = 5(y - 20) \bmod 26$

Main findings:

- The key depends only on the **mood**: $a = 2m + 1$, $b = 2m \pmod{26}$. The mood is sent *unencrypted*, so the key is effectively public.
- Server → Client is a plain affine cipher on upper-case letters. Client → Server first writes every character as two hex digits and then affine-encrypts the digits, so one character becomes two cipher letters.
- Every reply starts with the username (`FAATEH ...`), which is a free known-plaintext crib.
- Using only the recovered protocol, the server was rebuilt as an Ollama-backed `server.py`. The original client works with it, including at a mood that was never captured (Section 3.4).
- The two recommended fixes (TLS 1.3 and UNIX domain sockets) were **implemented and tested** (Section 4.3). The same attack fails against them.

1 Ciphertext Collection & Evidence

1.1 Offline lab environment

All work ran on one machine inside an isolated Linux environment (Ubuntu 24.04 under WSL2, NVIDIA RTX 3050 Ti). The provided `client` and `server` programs are Linux executables, so they were run there.

1. **Ollama installation.** Installed with the official script. The URL in the handout is missing the script path; the working command is:

```
curl -fsSL https://ollama.com/install.sh | sh      # Ollama 0.40.4, service on
    127.0.0.1:11434
```

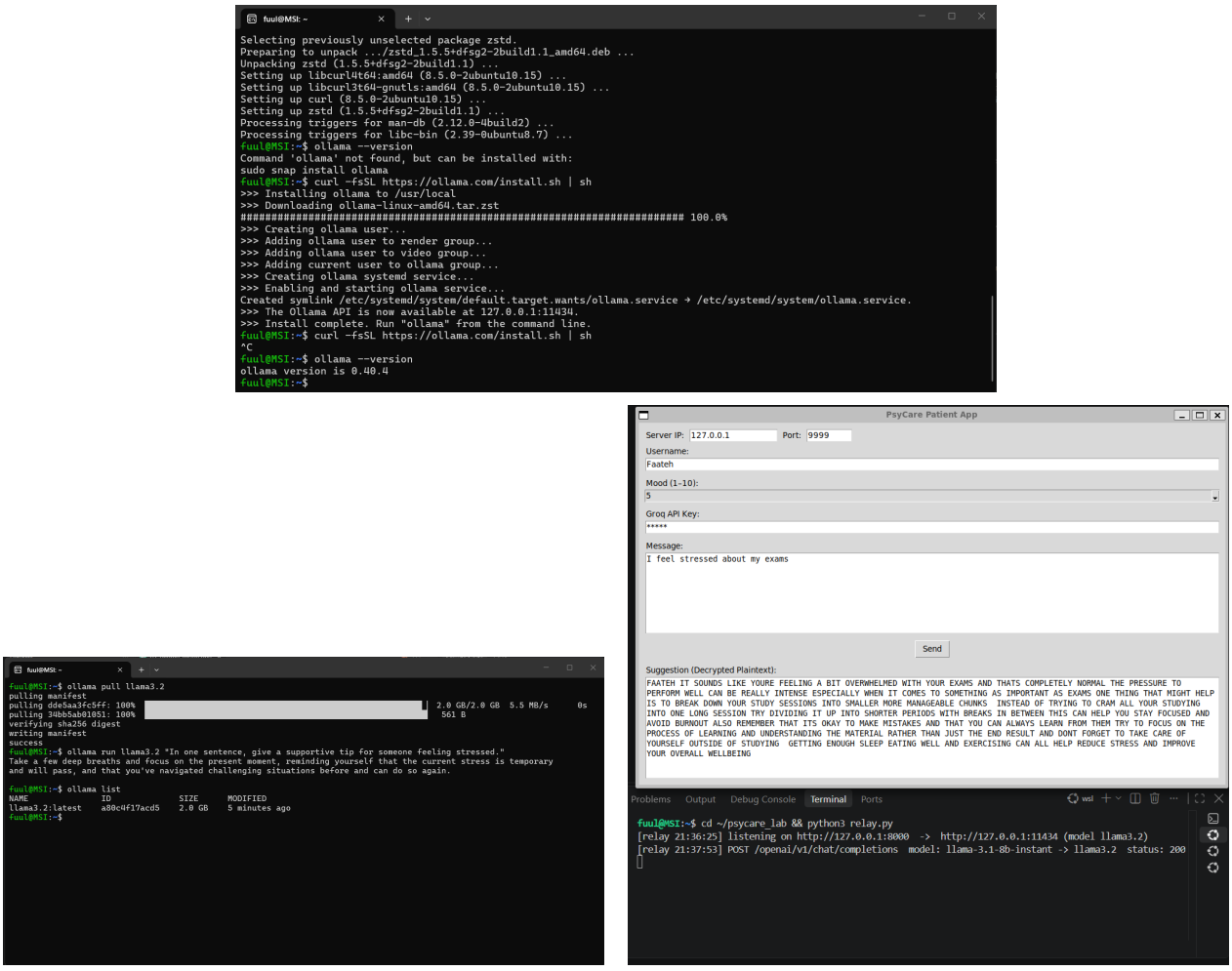
2. **Model provisioning.** `ollama pull llama3.2` (3B parameters, 2.0 GB), tested with a one-line prompt (Figure 1).
3. **Server binding.** The supplied `server` calls Groq's cloud API with the model `llama-3.1-8b-instant`, which Groq has since retired, so every request failed with HTTP 404. The handout says the server's functionality must not change, so the executable was left untouched. Instead, the Groq client library inside it was pointed at a small local relay (`relay.py`, submitted) through its standard `GROQ_BASE_URL` setting. The relay swaps the model name for `llama3.2` and forwards the request to Ollama:

```
client --(9999, encrypted)--> server --(8000)--> relay.py --(11434)--> Ollama
    llama3.2

python3 relay.py                                                    # tab 1
GROQ_BASE_URL=http://127.0.0.1:8000 ./server                        # tab 2: "[SERVER]
    Listening on 0.0.0.0:9999"
./client                                                            # tab 3: IP 127.0.0.1, port
     9999, user Faateh
```

The server listens on TCP port **9999** (not 5000 as suggested in the handout). The API-key field is ignored, as instructed; the dummy value `dummy` was typed.

All ciphertexts in this report were captured from this original, unmodified server, so the analysis attacks the real application. No server source was provided, so its protocol had to be recovered first. Once it was (Sections 2–3), the server was re-implemented as a standalone Ollama-backed `server.py` and validated against the original client (Section 3.4).



(a) Model downloaded and answering locally. **(b)** First end-to-end run; the relay logs `status: 200`.

Figure 1: Offline pipeline working. Top: Ollama installed (`ollama version is 0.40.4`).

1.2 Capture method

Traffic was captured on the loopback interface, filtered to the application port:

```
sudo tcpdump -i lo -w ~/psycare_lab/capture.pcap 'tcp port 9999'
```

The filter deliberately leaves out the server-to-relay hop (port 8000), so the capture contains only what an eavesdropper on the client–server channel sees. **50 packets** were captured across **5 TCP conversations**, one per session. Ciphertexts were extracted from the `.pcap` by a small Python script that reassembles each TCP stream, and cross-checked with Wireshark's *Follow TCP Stream* (Figure 3).

1.3 Test cases

The username was always `Faateh`, as instructed. Moods were varied so the effect of the mood could be studied, and two sessions share mood 5 so they could be compared.

Table 2: The five test sessions (concerns are synthetic).

Session	Mood	Ciphertexts	Concern typed into the client
1	5	C1, S1	I have not been sleeping well and I feel tired all the time
2	2	C2, S2	I feel stressed about my exams
3	5	C3, S3	My friends have been ignoring me lately and I feel lonely
4	8	C4, S4	I got a good grade but I still worry about the next semester
5	10	C5, S5	I feel great today and want tips to keep my motivation up

1.4 Message format on the wire

Each conversation is one JSON request followed by one JSON reply (session 1 shown, values shortened):

```
C->S {"username_enc": "CYYVYVJCYNYU", "mood": 5, "message_enc": "CFGKYUYVJYYNGKYI...", "
    api_key_enc": "YCJNYXYXJF"}
S->C {"ciphertext": "NKKLCJ UL AIWXRA BUQC OIWPC YIUXY LJPIWYJ K LIWYJ ..."}
```

Three things are visible before any cryptanalysis:

- the `mood` is sent in **plaintext**;
- Client → Server fields are twice as long as their plaintext (`Faateh`, 6 characters → 12 letters);
- Server → Client keeps spaces, so word lengths leak.

1.5 Ciphertext inventory

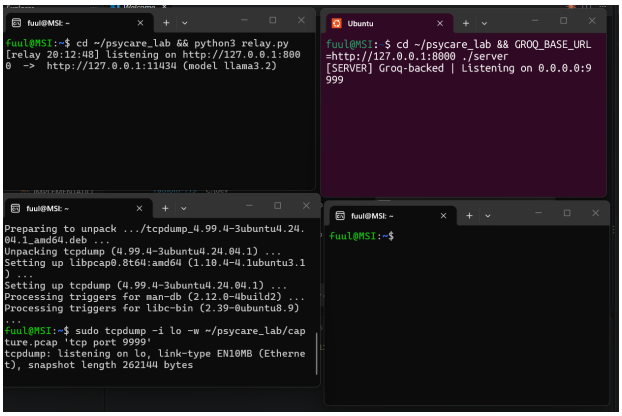
All ten ciphertexts are listed in full in `ciphertexts.csv`.

Table 3: Captured ciphertexts (first 40 characters).

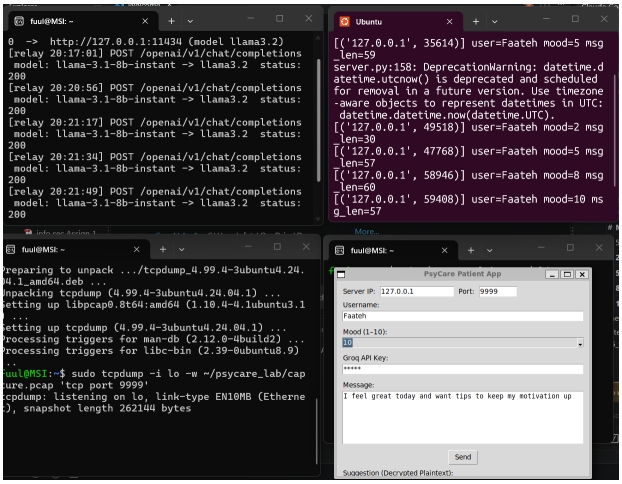
ID	Direction	Mood	Length	Ciphertext
C1	Client → Server	5	119	`CFGKYUYVJYYNGKYIYTJCGKYGYNYNYIGKJRYMYNYN...`
C2	Client → Server	2	60	`YXOEIIIDIDIMOENTNYNOIDNTNTIDIYOEIJIOIBND...`
C3	Client → Server	5	115	`CXJFGKYYJGYFYNYIYCJRGKYUYVJYYNGKYGYNYNYI...`
C4	Client → Server	8	121	`GNYQOFOLFGYQOHYQOFOLOLOGYQOFFYOHOGOXYQOY...`
C5	Client → Server	10	115	`ABKUQQQVQVQMKUQLLKQVQPLAKULAQXQAQPLBKUQP...`
S1	Server → Client	5	955	`NKKLCJ UL AIWXRA BUQC OIWPC YIUXY LJPIWYJ...`
S2	Server → Client	2	760	`DEEVYN S OER QYRQY VNEV UWALY DYYHSRI WFY...`
S3	Server → Client	5	1057	`NKKLCJ UL AIWXRA BUQC OIWPC NCCBUXY K VUL...`
S4	Server → Client	8	807	`XQQBGF WB KUSDPK VWEG IUSTG XGGVWDO Q HWB...`
S5	Server → Client	10	809	`VUUDAL GD ICYHFI RGWA ECYNA VAARGHQ GHKNAF...`

S1 and S3 (both mood 5) begin with the *same* 27 characters, while every other mood looks different. This was the first evidence that the key depends on the mood.

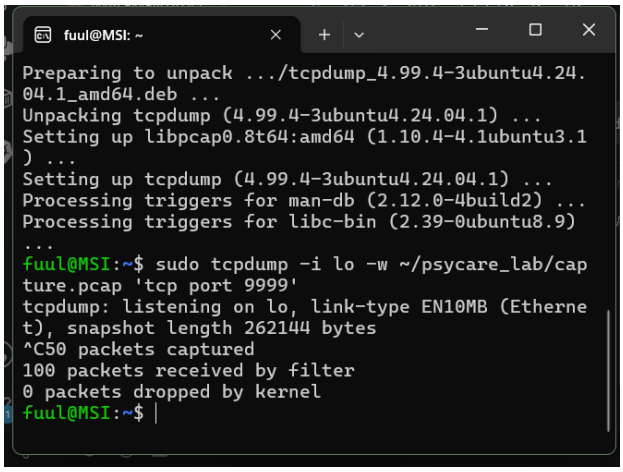
1.6 Capture evidence



(a) Relay, server and `tcpdump` running before the sessions.

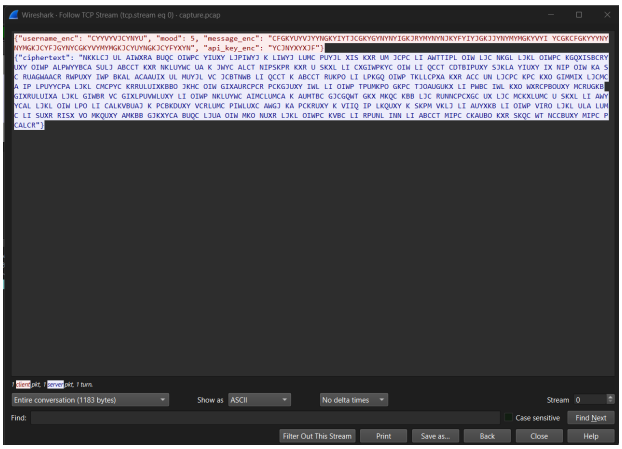


(b) All five sessions: server log shows moods 5, 2, 5, 8, 10.

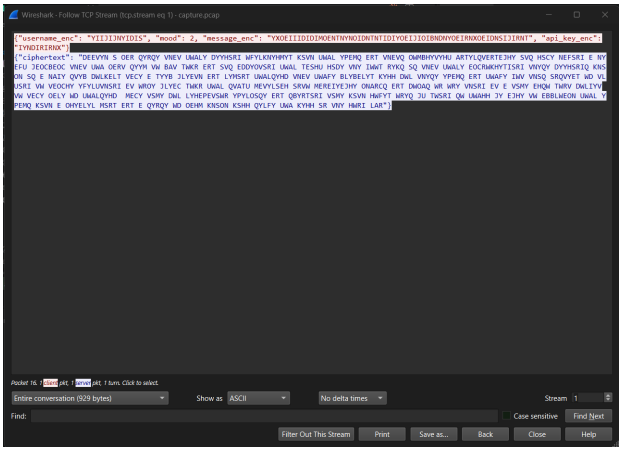


(c) Capture stopped: 50 packets captured, 0 dropped.

Figure 2: Capture of the five sessions on the loopback interface.



(a) Stream 0: session 1, mood 5.



(b) Stream 1: session 2, mood 2.

Figure 3: Wireshark *Follow TCP Stream* of the capture: red = Client → Server, blue = Server → Client.

2 Frequency Analysis & Key Recovery

2.1 Method

Letters are numbered A $= 0, \ldots,$ Z $= 25$. The affine cipher and its inverse are

$$E(x) = (ax + b) \bmod 26, \qquad D(y) = a^{-1}(y - b) \bmod 26,$$

which needs $\gcd(a, 26) = 1$, i.e. $a \in \{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$: only $12 \times 26 = 312$ keys.

Preprocessing. Only the letters A–Z are counted. JSON syntax, field names, spaces and the occasional stray space inside Client → Server ciphertexts are removed. Ciphertexts encrypted under the same mood are pooled (S1 with S3, C1 with C3), because Section 1 showed that they share a key.

Al-Kindi's method. Every affine key maps each plaintext letter to one fixed cipher letter, so letter frequencies survive encryption; they are only moved to different letters. The cipher letters are ranked by frequency and matched against English ranks (E 12.7%, T 9.1%, A 8.2%, O 7.5%, I 7.0%, N 6.7%, ...). Each guessed pair of mappings gives two linear congruences in a and b.

Testing candidates. Every solution with $\gcd(a, 26) = 1$ is used to decrypt the text, and the result is scored against English with

$$\chi^2 = \sum_{i=A}^{Z} \frac{(O_i - np_i)^2}{np_i},$$

where O_i is the observed count, n the number of letters and p_i the English frequency. Real English scores in the tens; wrong keys score in the hundreds or thousands. As an independent check, all 312 keys were also tried exhaustively.

The tables and charts were generated with `frequency_analysis.py` (helper `affine.py`); candidate keys were tested and scored with a companion script built on the same helper.

2.2 Server → Client (psychologist replies)

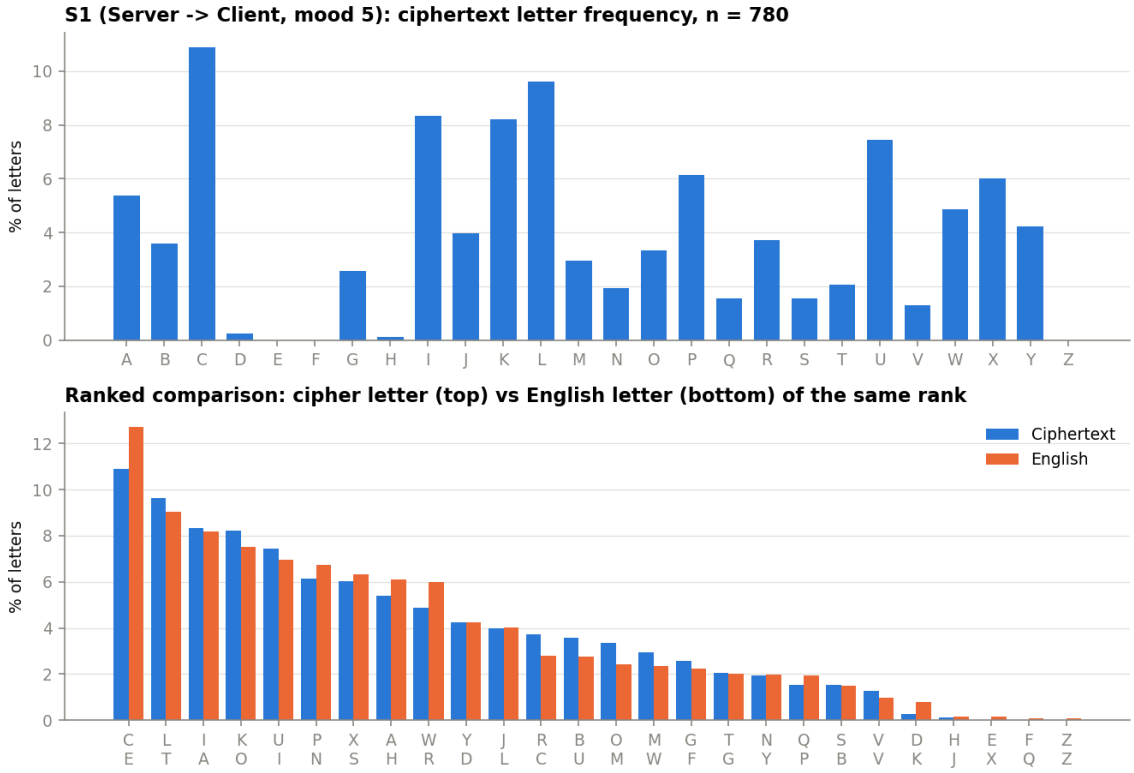


Figure 4: S1 (mood 5). Top: cipher letter frequencies. Bottom: cipher letters ranked against English letters of the same rank (cipher letter above, English letter below). `C` and `L` line up with `E` and `T`.

Table 4: S1 ranked frequencies ($n = 780$ letters) next to English. Tables for S2–S5 are in Appendix B.

Rank	Cipher	Count	%	English	%
1	C	85	10.90	E	12.70
2	L	75	9.62	T	9.06
3	I	65	8.33	A	8.17
4	K	64	8.21	O	7.51
5	U	58	7.44	I	6.97
6	P	48	6.15	N	6.75
7	X	47	6.03	S	6.33
8	A	42	5.38	H	6.09
9	W	38	4.87	R	5.99
10	Y	33	4.23	D	4.25
11	J	31	3.97	L	4.03
12	R	29	3.72	C	2.78
13	B	28	3.59	U	2.76
14	O	26	3.33	M	2.41
15	M	23	2.95	W	2.36
16	G	20	2.56	F	2.23
17	T	16	2.05	G	2.02
18	N	15	1.92	Y	1.97
19	Q	12	1.54	P	1.93
20	S	12	1.54	B	1.49
21	V	10	1.28	V	0.98
22	D	2	0.26	K	0.77
23	H	1	0.13	J	0.15
24	E	0	0.00	X	0.15
25	F	0	0.00	Q	0.10
26	Z	0	0.00	Z	0.07

The ciphertext distribution has the same shape as English: one letter clearly on top (`C` 10.9%), a group of letters at 6–10%, and a long tail of rare letters (`E`, `F`, `Z` almost absent). This is what a *monoalphabetic* substitution looks like, and it is exactly what Al-Kindi's method exploits.

Mood 5: full working (S1 + S3, 1640 letters)

Most frequent cipher letters: `C` 11.5%, `L` 9.9%, `I` 8.8%, `K` 7.7%, `U` 7.1%.

Hypothesis: E → `C` and T → `L` (the two most common letters in each).

$$\begin{aligned} 4a + b &\equiv 2 \pmod{26} && (\text{E} = 4 \to \texttt{C} = 2) \\ 19a + b &\equiv 11 \pmod{26} && (\text{T} = 19 \to \texttt{L} = 11) \\ (4 - 19)\, a &\equiv 2 - 11 && (\text{subtract}) \\ -15a \equiv -9 \;&\Rightarrow\; 11a \equiv 17 \pmod{26}. \end{aligned}$$

$\gcd(11, 26) = 1$, so 11 has an inverse. Extended Euclid:

$$\begin{aligned} 26 &= 2 \cdot 11 + 4, \qquad 11 = 2 \cdot 4 + 3, \qquad 4 = 1 \cdot 3 + 1; \\ 1 &= 4 - 3 = 4 - (11 - 2 \cdot 4) = 3 \cdot 4 - 11 \\ &= 3(26 - 2 \cdot 11) - 11 = 3 \cdot 26 - 7 \cdot 11. \end{aligned}$$

So $11^{-1} \equiv -7 \equiv 19 \pmod{26}$ (check: $11 \times 19 = 209 = 8 \cdot 26 + 1$). Then

$$a \equiv 19 \times 17 = 323 \equiv \mathbf{11}, \qquad b \equiv 2 - 4 \times 11 = -42 \equiv \mathbf{10} \pmod{26}.$$

$\gcd(11, 26) = 1$, so the key is valid, and decryption is $D(y) = 19\,(y - 10) \bmod 26$. The result reads `FAATEH IT SOUNDS LIKE YOURE GOING THROUGH A TOUGH TIME ...`, so the key is accepted.

Moods 2, 8 and 10

The same procedure was applied to each mood. When the first guess pairs E with A, the difference $(4-0)=4$ is *not* invertible mod 26 ($\gcd(4,26)=2$). The congruence is then divided through by 2 and solved mod 13, which gives two candidates; only the odd one is coprime with 26.

Mood 2 (S2, 622 letters). Top letters `Y` 12.9%, `E` 9.5%, `V` 8.2%. Best mapping: E → `Y`, A → `E`.

$$\begin{aligned} b &\equiv 4 && (\text{A} = 0 \to \texttt{E} = 4)\\ 4a + 4 \equiv 24 &\;\Rightarrow\; 4a \equiv 20 \pmod{26} &&\\ 2a \equiv 10 \pmod{13} &\;\Rightarrow\; a \equiv 5 \pmod{13} \;\Rightarrow\; a \in \{5, 18\} && \end{aligned}$$

Only $a=5$ is coprime with 26, so $(\mathbf{a},\mathbf{b}) = (\mathbf{5},\mathbf{4})$, with $5^{-1}=21$ ($5\cdot 21 = 105 = 4\cdot 26+1$).

Mood 8 (S4, 666 letters). Top letters `G` 11.6%, `B` 10.5%, `Q` 9.6%. Mapping E → `G`, T → `B`:

$$\begin{aligned} 4a + b \equiv 6, \qquad 19a + b \equiv 1 &\;\Rightarrow\; 11a \equiv 5 \pmod{26}\\ a \equiv 11^{-1} \times 5 = 19 \times 5 = 95 &\equiv 17\\ b \equiv 6 - 4 \times 17 = -62 &\equiv 16 \end{aligned}$$

So $(\mathbf{a},\mathbf{b}) = (\mathbf{17},\mathbf{16})$, with $17^{-1}=23$ ($17\cdot 23 = 391 = 15\cdot 26+1$).

Mood 10 (S5, 672 letters). Top letters `A` 11.8%, `U` 9.4%, `D` 8.9%. Mapping E → `A`, A → `U`:

$$\begin{aligned} b &\equiv 20 && (\text{A} = 0 \to \texttt{U} = 20)\\ 4a + 20 \equiv 0 &\;\Rightarrow\; 4a \equiv 6 \pmod{26} &&\\ 2a \equiv 3 \pmod{13} &\;\Rightarrow\; a \equiv 3\cdot 7 = 21 \equiv 8 \pmod{13} \;\Rightarrow\; a \in \{8, 21\} && \end{aligned}$$

(using $2^{-1} \equiv 7 \pmod{13}$). Only $a=21$ is coprime with 26, so $(\mathbf{a},\mathbf{b}) = (\mathbf{21},\mathbf{20})$, with $21^{-1}=5$ ($21\cdot 5 = 105 = 4\cdot 26+1$).

Candidate testing and confirmation

For each mood, the top cipher letter was mapped to E and the next five cipher letters were each tried as T, A, O, I and N: 25 hypotheses per mood. Every valid key was scored. The exhaustive 312-key search agrees in every case, and by a wide margin:

Table 5: Chosen key vs. the next-best of all 312 keys (lower χ^2 = closer to English).

Mood	Winning mappings	Key	χ^2 (key)	χ^2 (runner-up)
2	E→`Y`, A→`E`	(5, 4)	43.4	855.4
5	E→`C`, T→`L`	(11, 10)	87.5	2183.9
8	E→`G`, T→`B`	(17, 16)	39.5	940.8
10	E→`A`, A→`U`	(21, 20)	31.5	953.2

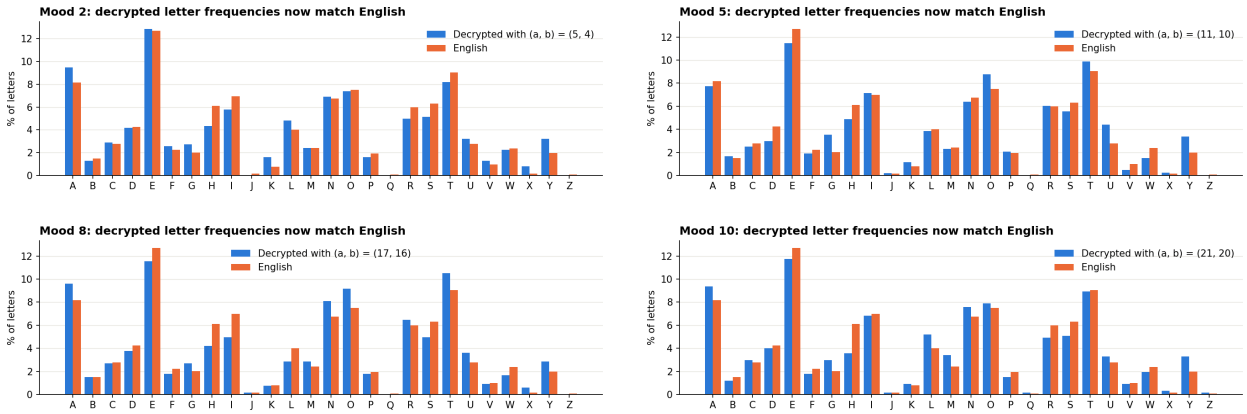


Figure 5: After decryption with the recovered keys, letter frequencies (blue) match standard English (orange).

2.3 Client → Server (patient requests)

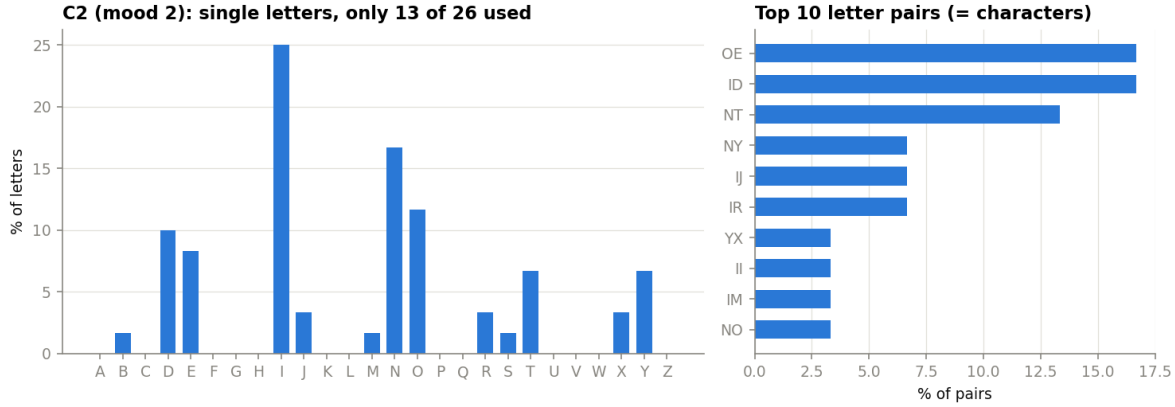


Figure 6: C2 (mood 2): only 13 of 26 letters ever appear (left). The letter pairs (right) behave like characters, with `OE` and `ID` the most common.

A normal English ciphertext would use 20–26 different letters. The Client → Server texts use only **13–14**, always have an even length, and `Faateh` (6 characters) becomes 12 letters. Each character is therefore encoded as **two symbols from a set of at most 16**, i.e. two **hexadecimal digits** of its ASCII code, and each digit $d \in \{0, \ldots, 15\}$ is then affine-encrypted as a letter. So the unit of frequency analysis is the *pair*.

Hypothesis. The most common character in English text is the **space**, roughly one character in five. Its ASCII code is `0x20`, i.e. the digits $(2, 0)$. If a pair `XY` is the space then

$$a \cdot 0 + b \equiv Y \;\Rightarrow\; b = Y, \qquad 2a + b \equiv X \;\Rightarrow\; 2a \equiv X - Y \pmod{26}.$$

Because $\gcd(2, 26) = 2$, this has *no* solution when $X - Y$ is odd, and otherwise two solutions a and $a + 13$, of which only the odd one is coprime with 26.

Mood 2 (C2). The top pairs are tied: `ID` and `OE` (16.7% each).

- `ID` as space: $b = 3$, $2a \equiv 8 - 3 = 5$, which is odd, so there is no solution. **Rejected.**
- `OE` as space: $b = 4$, $2a \equiv 14 - 4 = 10 \Rightarrow a \in \{5, 18\}$. $a = 18$ is even, so $(\mathbf{a}, \mathbf{b}) = (\mathbf{5}, \mathbf{4})$. Decrypting gives `I feel stressed about my exams`. **Accepted.**

The rejected pair `ID` is the letter `e` (`0x65`): $5 \cdot 6 + 4 = 34 \equiv 8 =$ `I` and $5 \cdot 5 + 4 = 29 \equiv 3 =$ `D`, which fits, since `e` is the most common English letter.

Table 6: Client → Server key recovery: the most common pair is the space (`0x20`).

Mood	Space pair	b	$2a \equiv$	Candidates	Key
2	`OE` (16.7%)	4	10	5, 18	(5, 4)
5	`GK` (19.0%)	10	22	11, 24	(11, 10)
8	`YQ` (20.0%)	16	8	4, 17	(17, 16)
10	`KU` (19.3%)	20	16	8, 21	(21, 20)

For every mood the even candidate is invalid, and the odd one decrypts to readable ASCII.

2.4 Key schedule

Putting both directions together:

Table 7: The same key is used in both directions and follows the mood exactly.

Mood m	Server → Client key	Client → Server key	$2m+1$	$2m$
2	(5, 4)	(5, 4)	5	4
5	(11, 10)	(11, 10)	11	10
8	(17, 16)	(17, 16)	17	16
10	(21, 20)	(21, 20)	21	20

$$\boxed{a = 2m + 1, \qquad b = 2m \pmod{26}}$$

This held for all four moods tested. One consequence is that mood $m = 6$ would give $a = 13$, which is not coprime with 26 and so cannot be decrypted uniquely. That mood was not tested. All sessions used the username `Faateh`, so this data shows no influence of the username on the key.

3 Decryption Results

3.1 Worked example: Server → Client (S1, mood 5)

Key $(11, 10)$, $a^{-1} = 19$, $D(y) = 19(y - 10) \bmod 26$. First word `NKKLCJ`:

Cipher	y	$19(y-10)$	mod 26	Plain
`N`	13	57	5	`F`
`K`	10	0	0	`A`
`K`	10	0	0	`A`
`L`	11	19	19	`T`
`C`	2	−152	4	`E`
`J`	9	−19	7	`H`

3.2 Worked example: Client → Server (C2, mood 2)

Key $(5, 4)$, $a^{-1} = 21$. Each letter gives a hex digit $d = 21(y - 4) \bmod 26$, and each pair of digits gives one character:

Letters	y	Digits $21(y-4) \bmod 26$	Hex	Character
`Y X`	24, 23	4, 9	`0x49`	`I`
`O E`	14, 4	2, 0	`0x20`	(space)
`I I`	8, 8	6, 6	`0x66`	`f`

3.3 All ciphertexts

Table 8: All 10 ciphertexts decrypted. Full replies are in Appendix A.

ID	Mood	Key	Plaintext
C1	5	(11, 10)	I have not been sleeping well and I feel tired all the time
C2	2	(5, 4)	I feel stressed about my exams
C3	5	(11, 10)	My friends have been ignoring me lately and I feel lonely
C4	8	(17, 16)	I got a good grade but I still worry about the next semester
C5	10	(21, 20)	I feel great today and want tips to keep my motivation up
S1	5	(11, 10)	`FAATEH IT SOUNDS LIKE YOURE GOING THROUGH A TOUGH TIME RIGHT NOW ...`
S2	2	(5, 4)	`FAATEH I CAN SENSE THAT YOURE FEELING OVERWHELMED WITH YOUR EXAMS ...`
S3	5	(11, 10)	`FAATEH IT SOUNDS LIKE YOURE FEELING A BIT ISOLATED RIGHT NOW ...`
S4	8	(17, 16)	`FAATEH IT SOUNDS LIKE YOURE FEELING A BIT ANXIOUS ABOUT NOT ...`
S5	10	(21, 20)	`FAATEH IT SOUNDS LIKE YOURE FEELING INCREDIBLY MOTIVATED ...`

Every decrypted request matches exactly what was typed into the client. The other encrypted request fields decrypt too:

Mood	`username_enc`	Plain	`api_key_enc`	Plain
2	`YIIJIJNYIDIS`	Faateh	`IYNDIRIRNX`	dummy
5	`CYYVYVJCYNYU`	Faateh	`YCJNYXYXJF`	dummy
8	`GOOHOHFGOXOW`	Faateh	`OGFXODODFN`	dummy
10	`AQQPQPLAQVQG`	Faateh	`QALVQHQHLB`	dummy

A real API key typed into this client would be exposed in exactly the same way.

A general decryption script was also written for any PsycoSupport ciphertext. Given the mood it applies $a = 2m+1$, $b = 2m$; without a mood it searches all 312 keys. Run blind over `ciphertexts.csv`, it recovered all 10 keys, and every one matched the key schedule.

3.4 Validation: an Ollama-backed `server.py`

As a final check that the recovered protocol is complete, the server was rebuilt from it alone as `server.py` (submitted). It listens on port 9999 and decrypts the request fields (hex layer, then $a = 2m + 1$, $b = 2m$). It asks the local `llama3.2` model for advice directly, without Groq or the relay. It then upper-cases the reply, keeps only letters and spaces, and affine-encrypts it with the same key.

Two tests were run:

1. **Replay of real traffic.** The exact request bytes of session 1, taken from the capture, were sent to `server.py`. It logged `user=Faateh mood=5 msg_len=59`, identical to the original server's log for that request, and its reply began `NKKLCJ ...`, the same opening as S1.
2. **The original client against the new server, at an unseen mood.** The unmodified `client` was used with mood **3**, a mood never captured, whose predicted key is $(a, b) = (7, 6)$.

The client decrypted the reply correctly (Figure 7), which confirms the key schedule on a new mood.

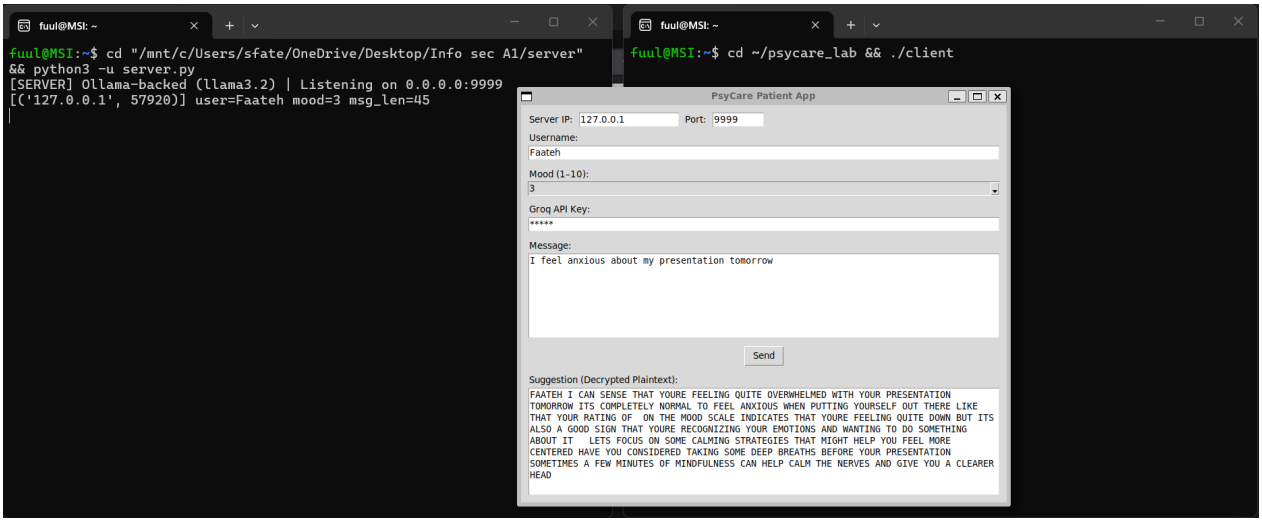


Figure 7: The original PsycoSupport client talking to the re-implemented, Ollama-backed `server.py` at mood 3. The digit "3" in the model's reply ("your rating of 3") is dropped because the protocol carries only letters and spaces, the same behaviour as the original server.

4 Security Analysis & Reflection

4.1 Why LLM output makes a classical cipher easy to break

- **Long, natural English.** Each reply is 600–1000 letters of ordinary prose, far more than the roughly 100 letters frequency analysis needs. The statistics converge on English (Figure 4).
- **Boilerplate and a fixed opening.** Four of the five replies begin `FAATEH IT SOUNDS LIKE YOURE FEELING`. The username is always the first word: a guaranteed *known plaintext*. Two letters are enough: from `NKKLCJ` = `FAATEH`, A→`K` gives $b = 10$, and F→`N` gives $5a + 10 \equiv 13 \Rightarrow a \equiv 3 \cdot 21 = 63 \equiv 11$. That is the full key without any statistics.
- **Formatting leaks structure.** The reply is upper-cased with punctuation removed, but spaces are kept, so word lengths, one-letter words (`A`, `I`) and repeated words (`YOU`, `THE`, `AND`) are visible.
- **Predictable requests.** Client messages are short, but the space character (about 20% of characters) and the fixed hex layout give two equations immediately.

4.2 Design flaws beyond the cipher

1. **Tiny keyspace:** 312 keys can be tried in milliseconds.
2. **The key is public:** it is derived from the mood ($a = 2m + 1$, $b = 2m$), and the mood is sent in plaintext in every request. Once the schedule is known, no cryptanalysis is needed at all.
3. **Same key in both directions** doubles the material available to an attacker for each key.
4. **The hex layer is encoding, not encryption.** It adds no secret, and its 16-symbol alphabet makes the ciphertext *more* recognisable.
5. **No integrity or authentication:** an attacker can change the mood or the ciphertext in transit and nothing detects it.

6. **Secrets under the weak cipher:** the API key is protected only by the same affine cipher.
7. **Wide binding:** the server listens on `0.0.0.0:9999`, i.e. on every network interface, not only on loopback.
8. **Plaintext internal hops:** the server sends the decrypted concern to the model unencrypted (via the relay here, or Groq originally).

4.3 Recommended alternatives (implemented and tested)

Both fixes recommended in the handout were built as a small, separate proof of concept. It does the same job (username, mood and concern in; advice from the local `llama3.2` out) without touching the lab programs.

1. **TLS 1.3 with a self-signed certificate** (`-mode tls`, port 9443). Python's `ssl` module with `minimum_version = TLSv1_3`. The client trusts only the server's certificate (pinned with `cafile`) and checks the hostname, so a man-in-the-middle with a different certificate is rejected. The negotiated cipher was `TLS_AES_256_GCM_SHA384` (authenticated encryption). A client limited to TLS 1.2 was refused with `PROTOCOL_VERSION`.
2. **UNIX domain socket** (`-mode unix`). The server listens on a socket *file*, `~/psycare_secure/psycare.sock`, with permissions `0600` (`srw----`). The traffic never enters the network stack, so packet sniffers cannot see it, and the operating system blocks other users from connecting. The certificate's private key is also `0600` and lives outside the project.

Same attack, repeated. `tcpdump` captured loopback (excluding Ollama's main port) while a demo script sent three messages over each channel. An analysis script then ran the eavesdropper's analysis on both captures:

Table 9: What an eavesdropper gets: original channel vs. TLS 1.3.

	Original (port 9999)	TLS 1.3 demo (port 9443)
Structure visible	JSON keys, `"mood": N`	none
Username / concern words in packets	no (but broken by analysis)	no
Symbols seen	23 letters A–Z (S1)	256 of 256 byte values
Shannon entropy	4.18 bits/letter (English ≈ 4.1)	7.96 bits/byte (max 8)
Frequency analysis	key recovered, all text read	nothing to map

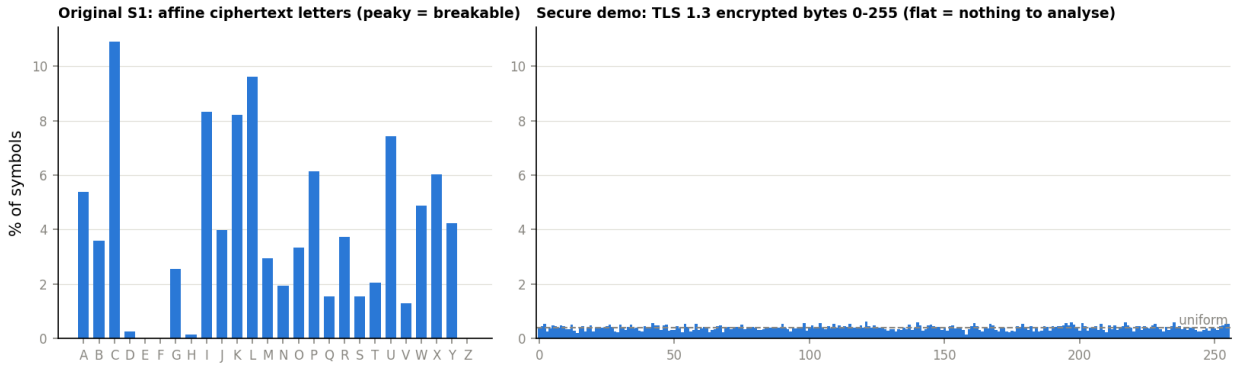


Figure 8: Left: affine ciphertext keeps the uneven, English-like shape that frequency analysis exploits. Right: TLS 1.3 encrypted bytes are essentially uniform (same vertical scale).

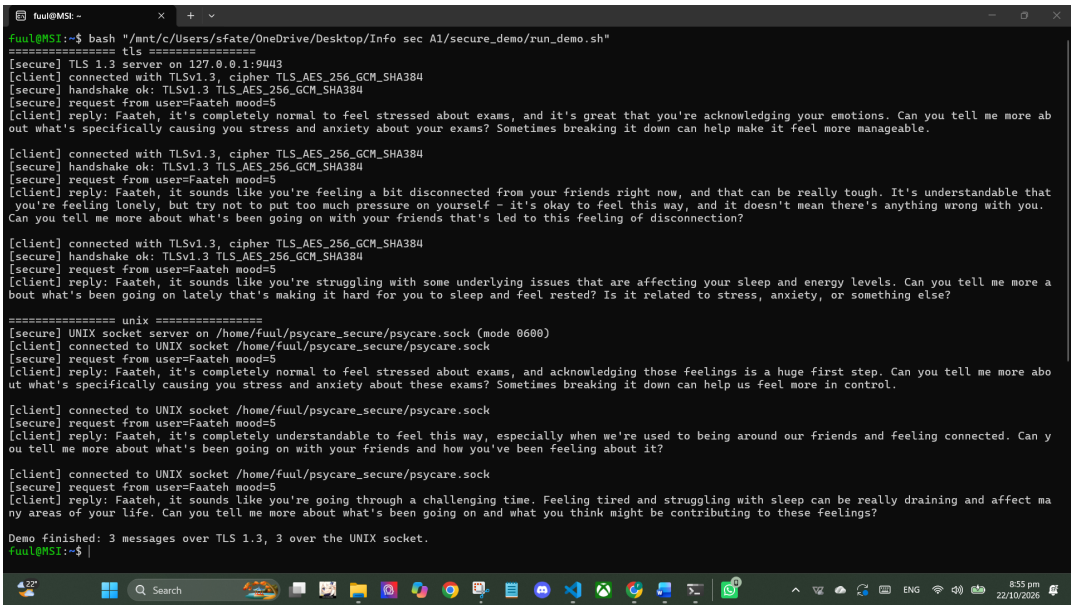


Figure 9: Secure demo: three exchanges over TLS 1.3 and three over the UNIX socket, captured with `tcpdump` running alongside.

The UNIX socket exchange never appeared in the capture: no packet contained the client's `"username"` request.

An important limitation found. The capture also contained 81,893 bytes on a random loopback port (39237): Ollama's *internal model runner*. Ollama forwards each prompt to it over plain HTTP, and the packets contained `Patient Faateh, mood 5/10: I feel stressed about my exams` in clear text. Protecting the client–server channel alone is therefore not enough. **Every hop that carries patient data needs protection**, including server→LLM. Options include running the model in-process, using only owner-restricted local IPC, or isolating the inference service in its own namespace or container. The same applies to the original lab, whose server→relay→Ollama hop is unencrypted.

Further recommendations: never derive keys from user-visible input, use authenticated encryption, do not send API keys from the client at all (keep them server-side), and bind services to `127.0.0.1` or a socket file rather than `0.0.0.0`.

4.4 Reflection

(a) Input influence. The *mood* completely determines the key. The same mood produced identical ciphertext openings (S1 and S3), and every other mood gave a different key ($a = 2m + 1$, $b = 2m$). The *concern* changes the plaintext and therefore the ciphertext, and it also shapes the reply: worried concerns gave replies that open with "it sounds like you're going through a tough time" or "feeling a bit anxious", while the positive one opened with "incredibly motivated". The *username* appears as the first word of every reply, so it leaks a known plaintext rather than changing the key. Only one username was used, as the handout requires.

(b) Communication direction. The two directions use the same key but different encodings. Server → Client encrypts upper-case letters and keeps spaces, so the text looks like scrambled English and gives way to single-letter analysis. Client → Server hex-encodes each character first, so it is twice as long, has no word breaks, uses at most 16 letters, and needs analysis of letter *pairs* instead. Requests are short (30–60 characters), while replies are long (600–1000 letters), so the replies carry far more statistical evidence. In practice the replies leak the key, and that key then unlocks the requests.

(c) Cryptographic behaviour. Frequency analysis showed that the cipher is monoalphabetic: every key just relabels the English distribution, which survives intact (Figure 4). With only 312 keys, a key that is derived from public data and the same in both directions, and highly regular LLM text, the scheme falls to both ciphertext-only and known-plaintext attacks within seconds. It gives a false sense of security: the traffic looks scrambled but protects nothing. A 9th-century technique was enough to break a 2026 application, which is why systems must use vetted modern protocols such as TLS 1.3 rather than home-made ciphers.

(d) Ethical considerations. This work was authorised and contained. It used only the instructor's PsycoSupport build, on a machine I control, over the loopback interface, with synthetic concerns and a dummy API key. No real patient or third party was involved. The same interception against a real application or real users without permission would be illegal (for example under Pakistan's Prevention of Electronic Crimes Act 2016 on unauthorised access and interception) and a serious breach of privacy. Mental-health conversations are among the most sensitive data a person can share, and exposing them could cause real harm. The skills are the same, so the deciding factors are *authorisation, scope and intent*. Penetration testers work under written permission, stay within agreed limits, and report weaknesses responsibly so they can be fixed, which is what Section 4.3 does for this application.

5 Submitted Files & Environment

File	Contents
`23I-0688-FaatehHaneef-A1.pdf`	This report.
`ciphertexts.csv`	All 10 captured ciphertexts: ID, session, direction, mood, length, ciphertext, and the encrypted username and API-key fields.
`server.py`	PsycoSupport server re-implemented with a local Ollama backend; same protocol, works with the original client (Section 3.4). Run: `python3 server.py`.
`relay.py`	Groq→Ollama bridge that let the unmodified original `server` run offline during capture (Section 1.1).
`frequency_analysis.py`	Generates the frequency tables and charts used in Section 2.
`affine.py`	Helper imported by `frequency_analysis.py`: English frequencies and affine-cipher maths.

Regenerating the charts (Python 3 with matplotlib; keep `affine.py` in the same folder):

```
python frequency_analysis.py ciphertexts.csv output     # -> output/figures/*.png,
    output/tables/*.csv
```

Environment.

- **Platform:** Ubuntu 24.04 under WSL2 (Windows 11), NVIDIA RTX 3050 Ti; Ollama 0.40.4 serving `llama3.2`; the supplied `client` and `server` executables, unmodified.
- **Capture:** `tcpdump` 4.99 on the loopback interface; Wireshark 4.6 for *Follow TCP Stream*.
- **Analysis:** Python 3 with matplotlib. Supporting scripts (not part of the requested submission) were also written for extracting ciphertexts from the capture, testing key candidates, general decryption and the TLS / UNIX-socket demonstration.

References

[1] Al-Kindi, *A Manuscript on Deciphering Cryptographic Messages*, 9th century. Summarised in S. Singh, *The Code Book*, Fourth Estate, 1999.

[2] R. E. Lewand, *Cryptological Mathematics*, Mathematical Association of America, 2000 (English letter frequencies).

[3] D. R. Stinson, M. B. Paterson, *Cryptography: Theory and Practice*, 4th ed., CRC Press, 2018 (affine cipher, extended Euclidean algorithm).

[4] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, IETF, 2018.

[5] Python Software Foundation, *ssl — TLS/SSL wrapper for socket objects*, Python 3 documentation.

[6] Linux man-pages, *unix(7) — sockets for local interprocess communication.*

[7] Ollama, documentation and OpenAI-compatible API, `https://github.com/ollama/ollama`.

[8] Government of Pakistan, *Prevention of Electronic Crimes Act*, 2016.

[9] M. A. Rehman, PsycoSupport lab repository, `https://github.com/maadilrehman/PsycoSupport`.

A Full decrypted replies

S1 (mood 5, key (11, 10))

FAATEH IT SOUNDS LIKE YOURE GOING THROUGH A TOUGH TIME RIGHT NOW AND IM HERE TO SUPPORT YOU THE FACT THAT YOURE ACKNOWLEDGING YOUR STRUGGLES WITH SLEEP AND FATIGUE IS A HUGE STEP FORWARD AND I WANT TO ENCOURAGE YOU TO KEEP EXPLORING WHATS GOING ON FOR YOU AS WE DISCUSSED DURING OUR LAST SESSION IT MIGHT BE HELPFUL TO KEEP A SLEEP DIARY TO TRACK YOUR PATTERNS AND SEE IF THERE ARE ANY COMMON THEMES OR TRIGGERS THAT EMERGE ADDITIONALLY HAVE YOU CONSIDERED REACHING OUT TO YOUR PRIMARY CARE PHYSICIAN TO RULE OUT ANY UNDERLYING MEDICAL CONDITIONS THAT COULD BE CONTRIBUTING TO YOUR FATIGUE SOMETIMES A SIMPLE CHECKUP CAN MAKE ALL THE DIFFERENCE IN THE MEANTIME I WANT TO SUGGEST THAT YOU TRY TO ESTABLISH A RELAXING BEDTIME ROUTINE SUCH AS READING A BOOK OR TAKING A WARM BATH TO SIGNAL TO YOUR BODY THAT ITS TIME TO WIND DOWN BY MAKING SMALL CHANGES LIKE THIS YOU MAY FIND THAT YOURE ABLE TO DRIFT OFF TO SLEEP MORE EASILY AND WAKE UP FEELING MORE RESTED

S2 (mood 2, key (5, 4))

FAATEH I CAN SENSE THAT YOURE FEELING OVERWHELMED WITH YOUR EXAMS AND THATS COMPLETELY UNDERSTANDABLE ITS LIKE HAVING A HEAVY BACKPACK THAT YOU CANT SEEM TO PUT DOWN AND ITS AFFECTING YOUR DAILY LIFE THE GOOD NEWS IS THAT YOURE ACKNOWLEDGING THESE FEELINGS WHICH IS A HUGE STEP FORWARD TAKE A DEEP BREATH AND REMIND YOURSELF THAT YOUVE PREPARED WELL FOR THESE EXAMS AND YOUVE GOT THIS INSTEAD OF TRYING TO TACKLE EVERYTHING AT ONCE BREAK DOWN YOUR STUDY MATERIAL INTO MANAGEABLE CHUNKS AND FOCUS ON ONE THING AT A TIME ALSO DONT FORGET TO TAKE CARE OF YOURSELF MAKE TIME FOR RELAXATION EXERCISE AND SPENDING TIME WITH LOVED ONES BY DOING SO YOULL BE ABLE TO APPROACH YOUR EXAMS WITH A CLEARER MIND AND A SENSE OF CALM WHICH WILL SERVE YOU WELL IN THE LONG RUN

S3 (mood 5, key (11, 10))

FAATEH IT SOUNDS LIKE YOURE FEELING A BIT ISOLATED RIGHT NOW AND THAT CAN BE REALLY TOUGH THIS MIGHT BE A GOOD OPPORTUNITY TO EXPLORE WHATS GOING ON BENEATH THE SURFACE OF YOUR FRIENDSHIPS ARE THERE ANY CHANGES IN YOUR LIFE OR RELATIONSHIPS THAT MIGHT BE CAUSING YOUR FRIENDS TO PULL BACK PERHAPS THERES SOMETHING YOUVE BEEN GOING THROUGH THAT YOU HAVENT SHARED WITH THEM OR MAYBE THEYRE DEALING WITH THEIR OWN ISSUES THAT ARE MAKING IT HARD FOR THEM TO CONNECT WITH YOU WHATEVER THE REASON ITS OKAY TO FEEL LONELY BUT TRY NOT TO TAKE IT PERSONALLY ITS POSSIBLE THAT YOUR FRIENDS ARE JUST BUSY OR PREOCCUPIED AND THAT THEY VALUE YOU AS A PERSON EVEN IF THEYRE NOT AS AVAILABLE AS THEY USED TO BE ONE THING YOU COULD TRY IS REACHING OUT TO A DIFFERENT FRIEND OR TWO JUST TO SEE IF YOU CAN RECONNECT WITH SOMEONE WHOS MORE AVAILABLE TO TALK THIS MIGHT ALSO BE A GOOD TIME TO THINK ABOUT EXPANDING YOUR SOCIAL CIRCLE OR JOINING A NEW GROUP OR CLUB THAT ALIGNS WITH YOUR INTERESTS SOMETIMES MEETING NEW PEOPLE CAN HELP YOU FEEL MORE CONNECTED AND LESS LONELY

S4 (mood 8, key (17, 16))

FAATEH IT SOUNDS LIKE YOURE FEELING A BIT ANXIOUS ABOUT NOT LETTING YOUR GUARD DOWN EVEN AFTER ACHIEVING A GOOD GRADE THIS IS A COMMON PHENOMENON ESPECIALLY AMONG STUDENTS WHO ARE DRIVEN TO EXCEL THE FACT THAT YOURE THINKING ABOUT THE NEXT SEMESTER ALREADY SUGGESTS THAT YOURE PROACTIVE AND RESPONSIBLE WHICH IS GREAT HOWEVER ITS ALSO IMPORTANT TO ACKNOWLEDGE THAT ITS OKAY TO FEEL SOME LEVEL OF UNCERTAINTY AND THAT IT DOESNT DEFINE YOUR WORTH AS A STUDENT OR A PERSON TRY TO FOCUS ON THE PRESENT MOMENT AND WHAT YOU CAN CONTROL RATHER

THAN WORRYING ABOUT WHAT MIGHT HAPPEN IN THE FUTURE REMEMBER THAT ONE SEMESTER AT A TIME AND DONT BE TOO HARD ON YOURSELF IF THINGS DONT GO EXACTLY AS PLANNED BY REFRAMING YOUR MINDSET YOU CAN WORK TOWARDS A MORE BALANCED AND MANAGEABLE APPROACH TO YOUR ACADEMIC JOURNEY

S5 (mood 10, key (21, 20))

FAATEH IT SOUNDS LIKE YOURE FEELING INCREDIBLY MOTIVATED AND ENERGIZED RIGHT NOW AND THATS WONDERFUL AS WE EXPLORE WAYS TO MAINTAIN THIS MOMENTUM I WANT TO ENCOURAGE YOU TO ACKNOWLEDGE AND APPRECIATE THE SMALL WINS ALONG THE WAY RATHER THAN FOCUSING SOLELY ON THE BIG GOALS TRY TO SAVOR THE TINY ACCOMPLISHMENTS THAT ADD UP TO MAKE A BIG DIFFERENCE FOR INSTANCE IF YOURE WORKING ON A PROJECT CELEBRATE EACH COMPLETED TASK OR MILESTONE NO MATTER HOW SMALL IT MAY SEEM THIS WILL HELP YOU STAY GROUNDED AND MOTIVATED EVEN WHEN THE GOING GETS TOUGH ADDITIONALLY MAKE SURE TO SCHEDULE TIME FOR SELFCARE AND RELAXATION AS A BALANCED LIFESTYLE IS CRUCIAL FOR SUSTAINING YOUR ENERGY AND ENTHUSIASM BY DOING SO YOULL BE BETTER EQUIPPED TO HANDLE ANY CHALLENGES THAT COME YOUR WAY AND MAINTAIN YOUR MOTIVATION OVER TIME

B Frequency tables and charts

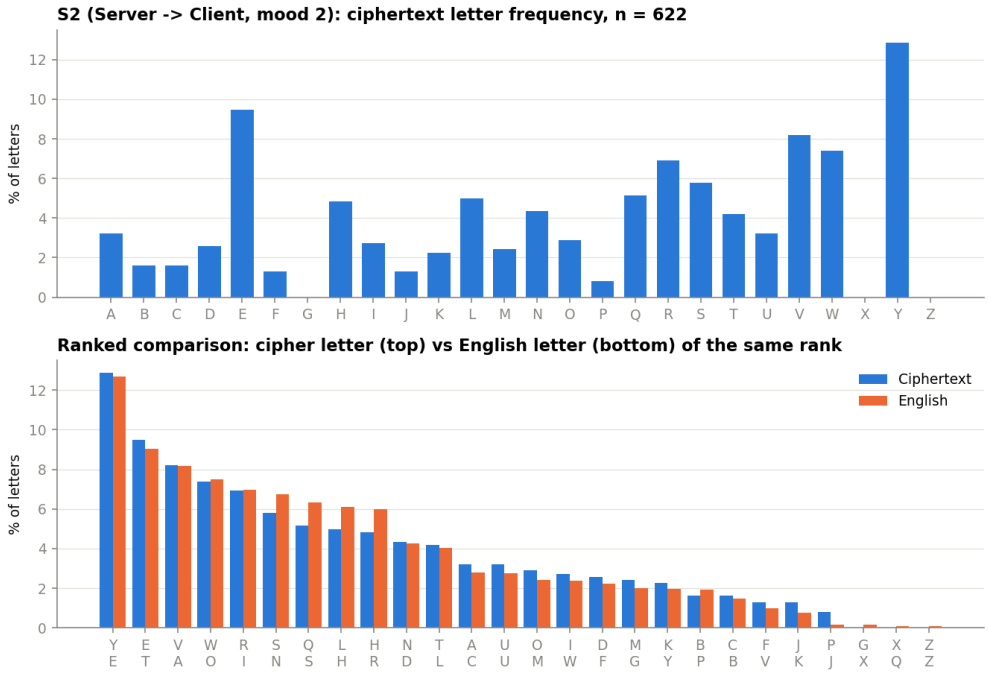


Figure 10: S2 (mood 2).

Table 10: S2 (mood 2) ranked frequencies, $n = 622$ letters.

Rank	Cipher	Count	%	English	%
1	Y	80	12.86	E	12.70
2	E	59	9.49	T	9.06
3	V	51	8.20	A	8.17
4	W	46	7.40	O	7.51
5	R	43	6.91	I	6.97
6	S	36	5.79	N	6.75
7	Q	32	5.14	S	6.33
8	L	31	4.98	H	6.09
9	H	30	4.82	R	5.99
10	N	27	4.34	D	4.25
11	T	26	4.18	L	4.03
12	A	20	3.22	C	2.78
13	U	20	3.22	U	2.76
14	O	18	2.89	M	2.41
15	I	17	2.73	W	2.36
16	D	16	2.57	F	2.23
17	M	15	2.41	G	2.02
18	K	14	2.25	Y	1.97
19	B	10	1.61	P	1.93
20	C	10	1.61	B	1.49
21	F	8	1.29	V	0.98
22	J	8	1.29	K	0.77
23	P	5	0.80	J	0.15
24	G	0	0.00	X	0.15
25	X	0	0.00	Q	0.10
26	Z	0	0.00	Z	0.07

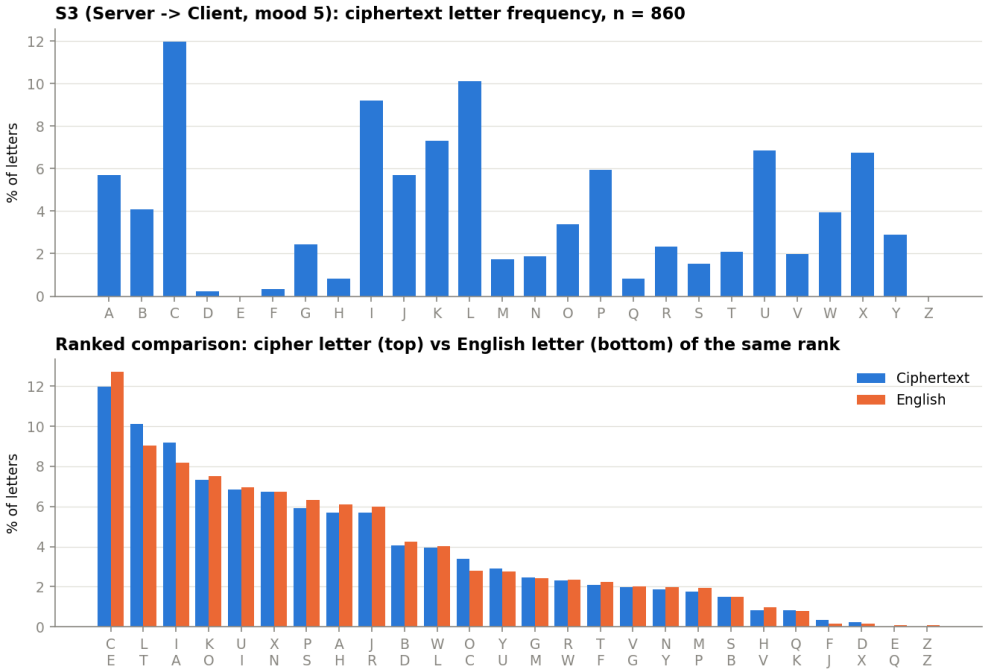


Figure 11: S3 (mood 5).

Table 11: S3 (mood 5) ranked frequencies, $n = 860$ letters.

Rank	Cipher	Count	%	English	%
1	C	103	11.98	E	12.70
2	L	87	10.12	T	9.06
3	I	79	9.19	A	8.17
4	K	63	7.33	O	7.51
5	U	59	6.86	I	6.97
6	X	58	6.74	N	6.75
7	P	51	5.93	S	6.33
8	A	49	5.70	H	6.09
9	J	49	5.70	R	5.99
10	B	35	4.07	D	4.25
11	W	34	3.95	L	4.03
12	O	29	3.37	C	2.78
13	Y	25	2.91	U	2.76
14	G	21	2.44	M	2.41
15	R	20	2.33	W	2.36
16	T	18	2.09	F	2.23
17	V	17	1.98	G	2.02
18	N	16	1.86	Y	1.97
19	M	15	1.74	P	1.93
20	S	13	1.51	B	1.49
21	H	7	0.81	V	0.98
22	Q	7	0.81	K	0.77
23	F	3	0.35	J	0.15
24	D	2	0.23	X	0.15
25	E	0	0.00	Q	0.10
26	Z	0	0.00	Z	0.07

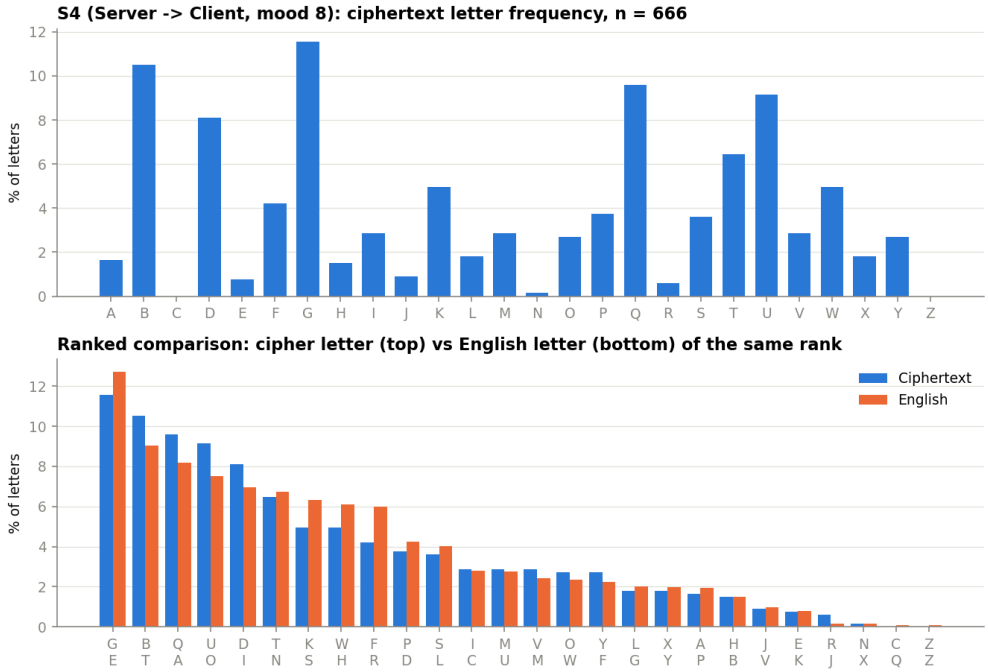


Figure 12: S4 (mood 8).

Table 12: S4 (mood 8) ranked frequencies, $n = 666$ letters.

Rank	Cipher	Count	%	English	%
1	G	77	11.56	E	12.70
2	B	70	10.51	T	9.06
3	Q	64	9.61	A	8.17
4	U	61	9.16	O	7.51
5	D	54	8.11	I	6.97
6	T	43	6.46	N	6.75
7	K	33	4.95	S	6.33
8	W	33	4.95	H	6.09
9	F	28	4.20	R	5.99
10	P	25	3.75	D	4.25
11	S	24	3.60	L	4.03
12	I	19	2.85	C	2.78
13	M	19	2.85	U	2.76
14	V	19	2.85	M	2.41
15	O	18	2.70	W	2.36
16	Y	18	2.70	F	2.23
17	L	12	1.80	G	2.02
18	X	12	1.80	Y	1.97
19	A	11	1.65	P	1.93
20	H	10	1.50	B	1.49
21	J	6	0.90	V	0.98
22	E	5	0.75	K	0.77
23	R	4	0.60	J	0.15
24	N	1	0.15	X	0.15
25	C	0	0.00	Q	0.10
26	Z	0	0.00	Z	0.07

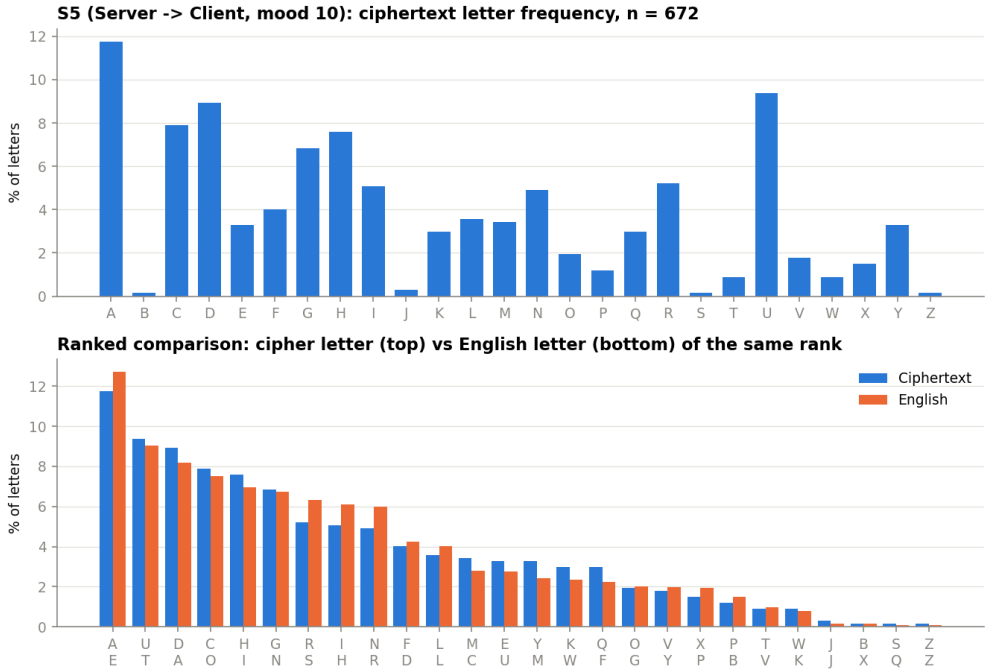


Figure 13: S5 (mood 10).

Table 13: S5 (mood 10) ranked frequencies, $n = 672$ letters.

Rank	Cipher	Count	%	English	%
1	A	79	11.76	E	12.70
2	U	63	9.38	T	9.06
3	D	60	8.93	A	8.17
4	C	53	7.89	O	7.51
5	H	51	7.59	I	6.97
6	G	46	6.85	N	6.75
7	R	35	5.21	S	6.33
8	I	34	5.06	H	6.09
9	N	33	4.91	R	5.99
10	F	27	4.02	D	4.25
11	L	24	3.57	L	4.03
12	M	23	3.42	C	2.78
13	E	22	3.27	U	2.76
14	Y	22	3.27	M	2.41
15	K	20	2.98	W	2.36
16	Q	20	2.98	F	2.23
17	O	13	1.93	G	2.02
18	V	12	1.79	Y	1.97
19	X	10	1.49	P	1.93
20	P	8	1.19	B	1.49
21	T	6	0.89	V	0.98
22	W	6	0.89	K	0.77
23	J	2	0.30	J	0.15
24	B	1	0.15	X	0.15
25	S	1	0.15	Q	0.10
26	Z	1	0.15	Z	0.07

Client → Server charts

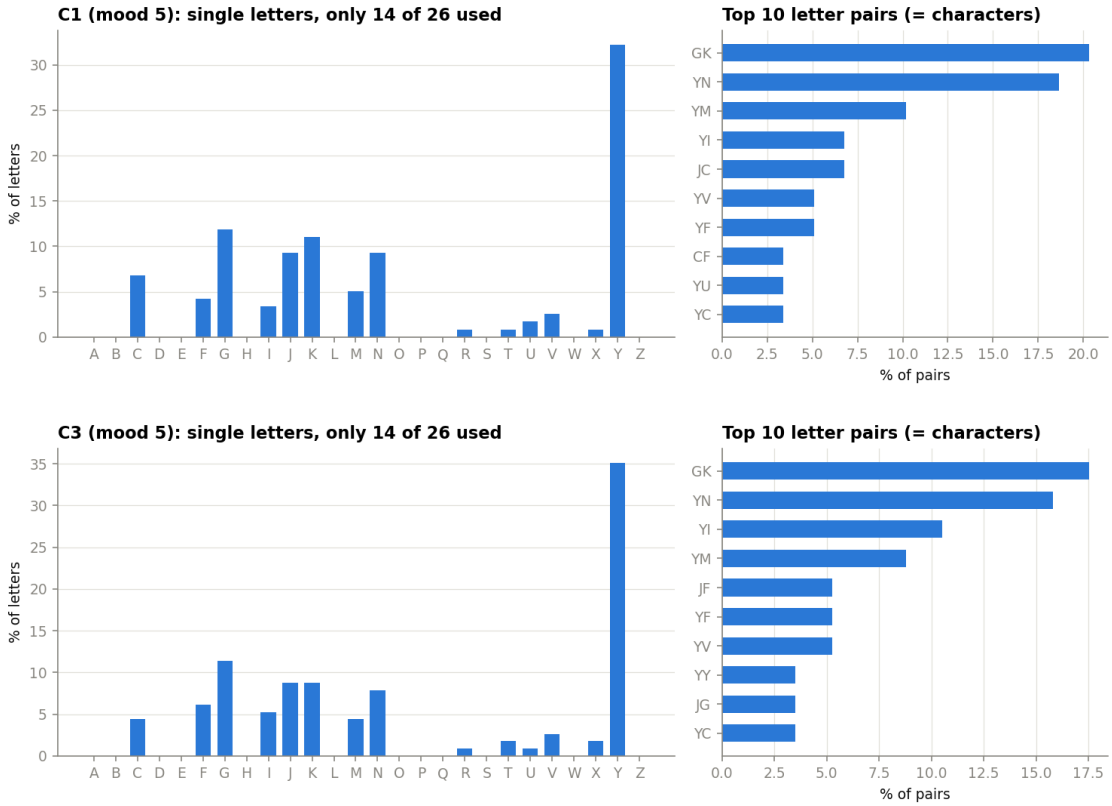


Figure 14: C1 and C3 (mood 5).

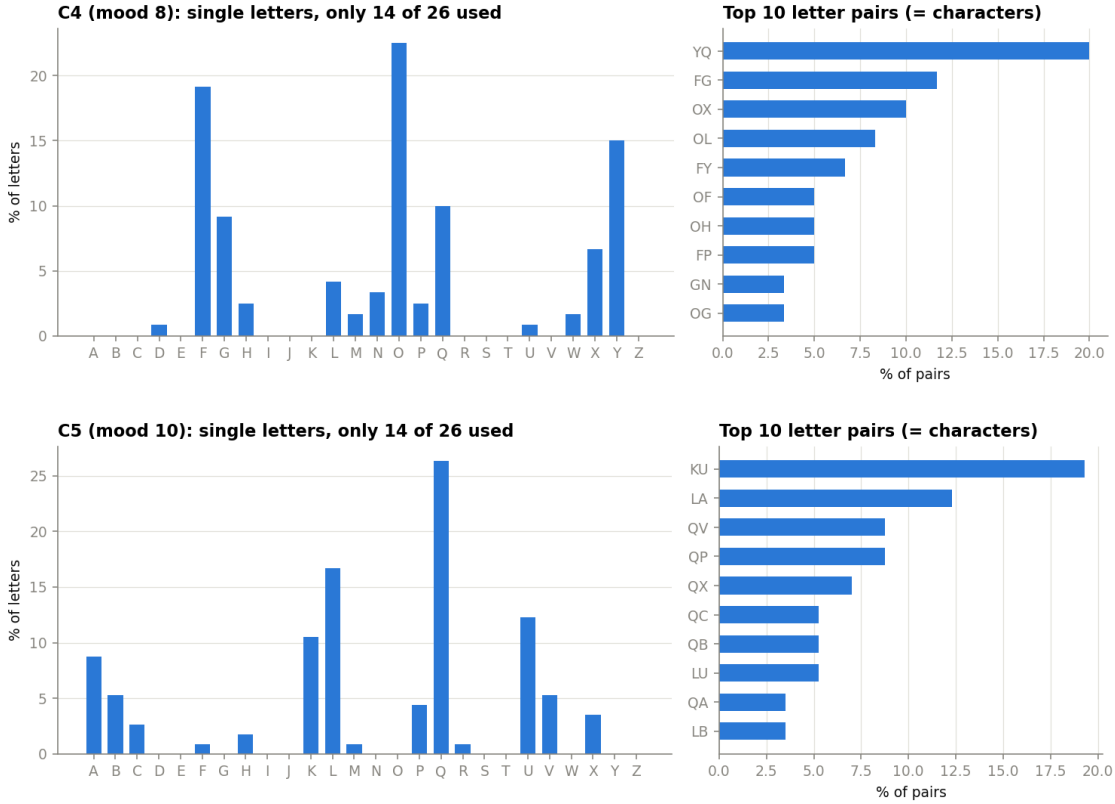


Figure 15: C4 (mood 8) and C5 (mood 10).